Unit – I

UNIT-1: Overview of Programming and Introduction to C Language

Introduction to Computers and Problem Solving: What is computer, Introduction to components of a computer: Hardware, Software, Programming Language Types, Compiler, Problem Solving Aspect, Top-Down Design.
Idea of Algorithm: steps to solve logical and numerical problems. Representation of Algorithm, Flowchart with examples.
From algorithms to programs: source code, variables, data types and memory locations, Syntax and Logical Errors in compilation, object and executable code.

Learning outcomes:
At the end of this unit, the students will be able to
1. Use C basic concepts to write simple C programs.
2. Test and execute the programs and correct syntax and logical errors.
3. Translate algorithms into programs.

What is Computer

The word computer is derived from latin word "Computare" is used for performing computations or calculations.

Definition of Computer

A computer is an electronic device that manipulates information, or data. It has the ability to store, retrieve, and process data.

Or

A Computer is an electronic device that is used for information processing. It accepts data/information, stores in its memory processes it and gives results to the user.

Or

The computer is an electronic device which converts raw data into valid or meaningful information.

Basic operations of a computer

The computer performs the following, five basic operations to carry out any task.

- ✓ **Input:** It is the process capturing or acquiring the information, or it is the process of accepting data or information, by using input the computer can do any process.
- ✓ **Process:** It is the transformation process to convert the input into output.
- ✓ **Output:** It is the result, which comes from the transformation process or it is the outcome of the process.
- ✓ **Storing:** It is the process of storing and retaining the data or information, so that the users can retain and retrieve it whenever required.
- ✓ **Controlling:** It is the process of directing the manner and sequence in which all the operations are to be performed.

Unit – I

Basic Computer Organization

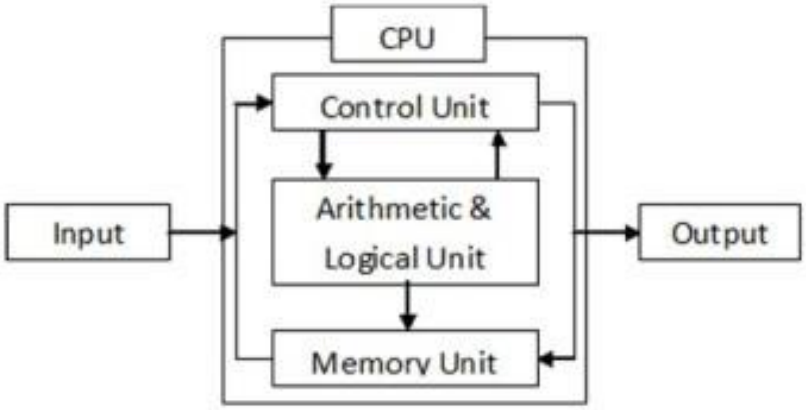


Fig. Block Diagram of Computer

The architecture of the computer has not changed, but the technology used to accomplish those operations may vary from one computer to another computer. In 1950's it took a room full of vacuum tubes and equipment's to perform the tasks that are now replaced with a single chip. The block diagram of the computer system has the following three units, each functional unit corresponds to their basic operations performed as described in detail.

- ✓ Input unit
- ✓ Central Processing unit (CPU)
- ✓ Output unit

- **Input**
  - All the data received by the computer goes through the input unit. The input unit comprises different devices like a mouse, keyboard, scanner, etc.
  - The data that is to be processed is put through the input unit. The computer accepts the raw data in binary form. It then processes the data and produces the desired output.
  - The three major functions of the input unit are
    - Take the data to be processed by the user.
    - Convert the given data into machine-readable form.
    - And then, transmit the converted data into the main memory of the computer.

- **CPU – Central Processing Unit**
  - Central Processing Unit (CPU) is the brain of the computer. It works the same way a human brain works. As the brain controls all human activities, similarly the CPU controls all the tasks. Moreover, the CPU conducts all the arithmetical and logical operations in the computer. The CPU comprises of three units, namely – ALU (Arithmetic and Logic Unit), Memory Unit and Control Unit.
  - **ALU – Arithmetic Logic Unit**
    - The Arithmetic and Logic Unit is made of two terms, arithmetic and logic. There are two primary functions that this unit performs.

- Data is inserted through the input unit into the primary memory. Performs the basic arithmetical operation on it. Like addition, subtraction, multiplication, and division. It performs all sorts of calculations required on the data. Then sends back data to the storage.
- The unit is also responsible for performing logical operations like AND, OR, Equal to, less than, etc. In addition to this it conducts merging, sorting, and selection of the given data.

- **Control Unit**
  - The control unit as the name suggests is the controller of all the activities/tasks and operations. All this is performed inside the computer.
  - The memory unit sends a set of instructions to the control unit. Then the control unit in turn converts those instructions. After that these instructions are converted to control signals.
  - These control signals help in prioritizing and scheduling activities. Thus, the control unit coordinates the tasks inside the computer in sync with the input and output units.
- **Memory Unit**
  - Memory is the part of computer which holds data for processing and other information. It is also called as Main Memory. It is a volatile memory i.e., whenever the power is lost, the stored information will be erased, to avoid this, we can store in secondary memory.
  - There are two types of computer memory
    - ✓ **Primary memory:** This type of memory cannot store a vast amount of data. Therefore, it is only used to store recent data. The data stored in this is temporary. It can get erased once the power is switched off. Therefore, is also called temporary memory or main memory. **Secondary memory:** It is permanent storage device. It is also called permanent memory or auxiliary memory. The hard disk is an example of secondary memory. Even in a power failure data does not get erased easily.

- **Output**
  - The output unit displays the data either in the form of a soft copy or a hard copy. The printer is for the hard copy. The monitor is for the display. The output unit accepts the data in binary form from the computer. It then converts it into a readable form for the user. Some more commonly used output devices printers, monitors, projectors, etc.

Various Components of Computer

A computer is a system made of two major components. They are hardware and software

- The computer hardware is the physical equipment.
- The software is the collection of programs (set of instructions) that allow the hardware to do its tasks.

Unit – I

Computer Hardware

The hardware components are classified into Input devices, Output devices, System Components and Storage devices.

- **Input devices:**
  - Keyboard
    - Arrangement of buttons or keys to act as mechanical levers or electronic switches.
    - Keyboard keys (buttons) typically have a set of characters engraved or printed on them.
    - letters, numerals, punctuation marks, and symbols, the typing keys also include Shift, Caps Lock, Tab, Enter, the Spacebar, and Backspace.
    - Each press of a key typically corresponds to a single written symbol.
    - However, producing some symbols may require pressing and holding several keys simultaneously or in sequence.
  - Mouse
    - Hand-held pointing device that detects two-dimensional motion relative to a surface.
    - This motion is typically translated into the motion of the pointer (called a cursor) on a display, which allows a smooth control of the GUI (Graphical User Interface) of a computer.
  - Microphone
    - Record music or sound, the information (a record) is stored on the computer.
    - Translates sound vibrations in the air into electronic signals.
  - Scanner
    - Captures images from photographic prints, posters, magazine pages and similar sources for computer editing and display.
    - Converting the image on the document into digital information that can be stored on a computer through optical character recognition (OCR).
  - Webcam
    - A webcam is a video camera which is designed to record or stream to a computer or computer network.
    - Primarily used in video telephony, live streaming and social media, and security.
  - Touchscreen
    - A touch screen is a computer display screen that is sensitive to human touch, allowing a user to interact with the computer by touching pictures or words on the screen.
  - Trackball
    - A trackball is a pointing device consisting of a ball held by a socket containing sensors to detect a rotation of the ball about two axes.
- **Output devices:**
  - Monitor
    - It also known as a CRT
    - It displays images in pictorial form on the screen.
    - Types of Monitors
      - LCD monitor.
      - LED monitor.

Unit – I

- OLED monitor.
- CRT monitor.
- Plasma monitor.

- Printer
  - It allows a user to print items on paper, such as letters and pictures.
  - Types of Printers
    - Dot Matrix Printers.
    - Laser Printers.
    - Thermal Printer.
    - Ink Printers.
    - 3D Printers.
- Plotter
  - It makes line drawings on paper with one or more automated pens.
  - It capable of producing graphs, drawings, charts, maps etc..
- Projector
  - It is an Optical device that projects an image onto a surface.
- Speaker or Headphone
  - It converts electromagnetic waves into sound waves.

- **System Components:**
  - Mother Board
    - A motherboard is the main printed circuit board (PCB) in a computer. The motherboard is a computer's central communications backbone connectivity point, through which all components and external peripherals connect.
  - CPU (Microprocessor or chip)
    - The CPU is the brain of a computer, containing all the circuitry needed to process input, store data, and output results. The CPU is constantly following instructions of computer programs that tell it which data to process and how to process it. Without a CPU, we could not run programs on a computer.
  - CMOS Battery
    - The CMOS (complementary metal oxide semiconductor) battery, also called coin-cell battery on the motherboard helps the BIOS or UEFI to store the hardware configuration settings.
  - Heatsink
    - A heat sink is a piece of metal that sits on top of a computer chip such as a CPU. Heat sinks are widely used for cooling parts and components that produce heat while in use.

- **Storage Devices:**
  - Hard disk
    - It is also called hard disk drive or hard drive or magnetic storage medium for a computer. Hard disks are flat circular plates made of aluminum or glass and coated with a magnetic material. Hard disks for personal computers can store huge amount of information.

- RAM (Random Access Memory)
  - It is the internal memory of the CPU for storing data, program, and program result. It is a read/write memory which stores data until the machine is working. RAM is volatile, i.e. data stored in it is lost when we switch off the computer or if there is a power failure.
- ROM (Read Only Memory)
  - The memory from which we can only read but cannot write on it. This type of memory is non-volatile. The information is stored permanently in such memories during manufacture. A ROM stores such instructions that are required to start a computer. This operation is referred to as bootstrap.
- Pen drive
  - A pen drive is a portable Universal Serial Bus (USB) flash memory device for storing and transferring audio, video, and data files from a computer.
- CD / DVD
  - A CD (Compact Disc) and DVD (Digital Versatile Disc) are optical storage media used to store digital data, such as music, videos, software programs, and documents.
- Floppy Disk
  - A floppy disk is a disk storage medium composed of a disk of thin and flexible magnetic storage medium encased in a rectangular plastic carrier.

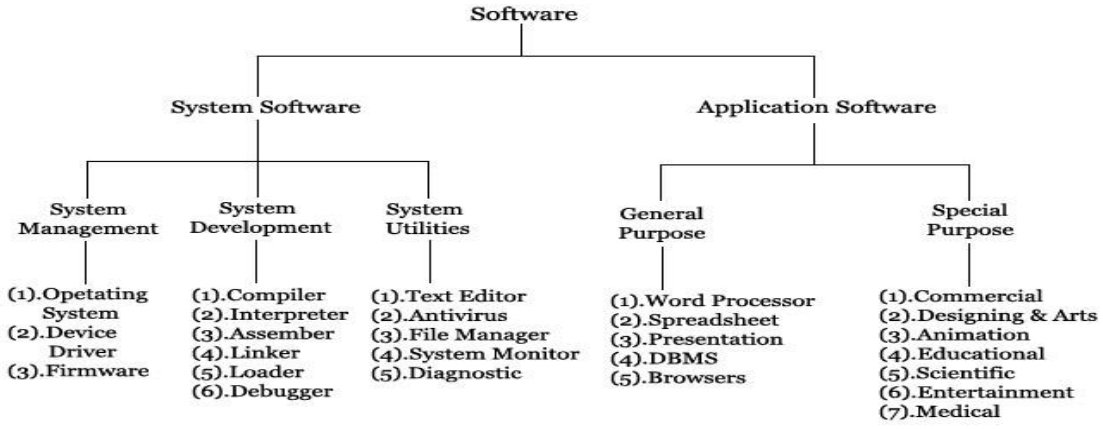
Computer Software

- Software is a set of instructions, data or programs used to operate computers and execute specific tasks. It is the opposite of hardware, which describes the physical aspects of a computer. Software is a generic term used to refer to applications, scripts and programs that run on a device.

Or

- Software refers to the instructions or programs that tell the hardware what to do. It is responsible for controlling, integrating and managing the hardware components of a computer and to accomplish specific tasks. In other words, software tells the computer what to do and how to do it.

- **Types of software:**



Unit – I

- **System Software**
  - System software acts as an **interface** between the hardware of the computer and the software applications.
  - The purpose of this software is to help the user to **run the computer** system.
  - System software consists of several programs, which are directly responsible for **controlling, integrating and managing** the individual hardware components of the computer system.
  - Some examples are
    1. System Management Software (Operating System, Device Drivers)
    2. System Development Software (Language Translators)
    3. System Utilities

  - **Operating System**
    - Operating system (OS) is an **interface between hardware and user**.
    - Operating system is the **first layer of software** loaded into computer memory.
    - These **common core service** include
      1. Disk access
      2. Memory management
      3. Configuring and controlling peripheral devices
      4. Managing essential file operations, including formatting or copying disks, and renaming or deleting files
      5. Monitoring system performance (Security management)
      6. Providing a user interface
    - Operating system ensures that different programs executing at the same time do not interfere with each other.
    - Examples of Operating systems are
      - ✓ DOS
      - ✓ Windows
      - ✓ Unix
      - ✓ Linux
    - Features of OS
      1. Process Management (program execution)
      2. Memory Management
      3. File management
      4. Device Management
      5. Security management
      6. User Interface

  - **Device Driver**
    - **Device driver** or **software driver** are system programs which are responsible for **proper functioning of devices**.
    - Every device such as printer, monitor, mouse or keyboard, has a driver associated with it for its proper functioning.
    - Whenever a new device is added to the computer system, a new **device driver (Software for particular device) must be installed before the device is used**.

Unit – I

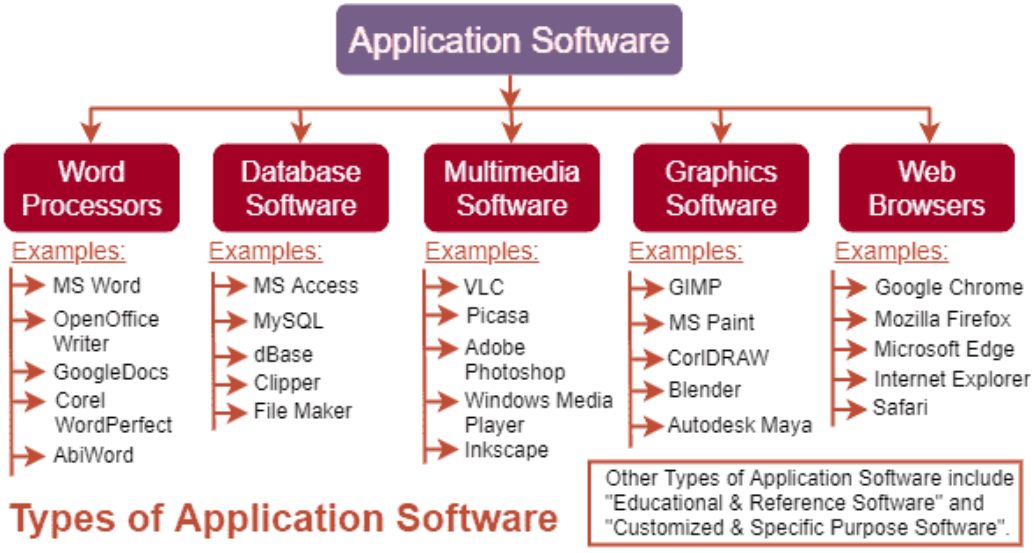
- In modern operating system, most hardware drivers such as keyboard **driver come with the operating system**
- A driver act like a **translator** between the device and program that uses the device.

- **Language Translator**
  - Computer can understand a language consisting of 0s and 1s called **machine language**
  - Special programming languages called **high- level language** were developed that resemble natural languages like English.
  - Therefore, **tool** was required which could translate a program written in a programming language to machine language.
  - Depending on the programming language used, language translators are divided into three major categories.
    1. Compiler
    2. Interpreter
    3. Assembler
  - **Compiler**
    - The program written in any **high-level language is converted into machine language** using a compiler.
  - **Interpreter**
    - The program written in any **high-level language is converted into machine language** using an Interpreter.
    - Interpreter **analyses and executes the source code in line-by-line manner,** without looking at the entire program.
  - **Assembler**
    - Assembly language is fundamentally a **symbolic representation of machine code**.
    - The **assembly language program translated into machine code** is called as **assembler.**

- **System Utility:**
  - System utility program perform day to day task **related to the maintenance** of the computer system.
  - They are used to **support, enhance and secure** existing programs and data in the computer system.
  - Some utility programs are usually provided along with the operating system, some are free while some need to be purchased from the 3rd party vendors.
    - **Ex. For system utilitie**s:
      1. Disk checkers
      2. Disk cleaners
      3. Antivirus utilities
      4. Registry cleaners
      5. Network managers

Unit – I

- **Application software**
  - Application software is used to accomplish **specific tasks**.
  - **General Purpose Software:**
    - This software is purchased from a software developer that can be used for more than one application. Example for general purpose software includes, word processors, database management systems and computer aided design systems. They are labeled general purpose because they can solve in a variety of user computing problems.
  - **Application specific software:**
    - This software can be used only for its intended purpose. A general ledger system used by accountants is an example. They can be used only for the task for which they were designed. They cannot be used for another task.
  - Some of the most commonly used application software are
    - Word processors
    - Spreadsheets
    - Image editors
    - Database management systems
    - Presentation Applications
    - Desktop Publishing Software



Programming Language

- A programming language is a set of instructions written by a programmer to deliver instructions to the computer to perform and accomplish a task. This set of instructions is usually viewed as incomprehensible code structured following a definite programming language syntax.

Or

- A programming language is a set of instructions and syntax used to create software programs.

Unit – I

- **Types of Programming Languages**
  - There are three types of programming languages. They are machine language, assembly language, and high-level language.
  - **Machine language:**
    - It is easier for the computer to understand but harder for the programmer to understand. It is composed of digital binary numbers and looks like a very long sequence of zeros and ones.
    - **Example**

```
00000000101000010000000000011000
00000000100011100001100000100001
10001100011000100000000000000000
10001100111100100000000000000100
10101100111100100000000000000000
10101100011000100000000000000100
00000011111000000000000000001000
```

  - **Assembly Language:**
    - An assembly language is a type of low-level programming language that is intended to communicate directly with a computer's hardware. Unlike machine language, which consists hexadecimal characters (Mnemonics), assembly languages are designed to be readable by humans.
    - **Example**

```
MVI A, #20
MVI B, #40
ADD A, B
STA 2042
HLT
```

  - **High Language:**
    - A high-level language (HLL) is a programming language such as C, FORTRAN or Pascal that enables a programmer to write programs that are more or less independent of a particular type of computer.
    - High-level languages are human-friendly. They are, thus, very easy to understand and learn by any programmer.
    - **Example (C Programming)**

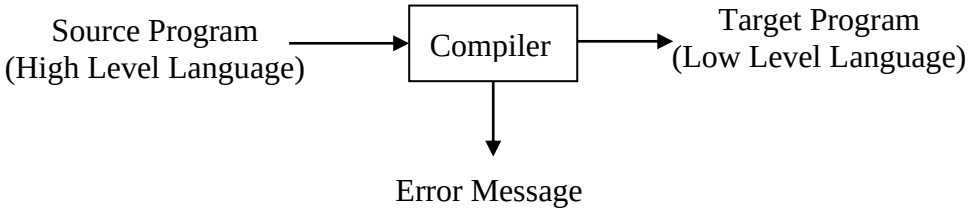
```
#include<stdio.h>
void main()
{
        int a,b,c;
        scanf("%d%d", &a,&b);
        c=a+b;
        printf("Sum = %d", c)
}
```

Unit – I

High-Level Language	Low-Level Language
High-level languages are near to human languages.	Low-level languages are far from human languages.
High-level languages do not provide much facility at hardware level.	Low-level languages provide facility to write programs at hardware level.
Deep knowledge of hardware is not required to write programs.	Deep knowledge of hardware is required to write programs.
These languages are normally used to write application programs.	These languages are normally used to write hardware programs.

Compiler

- A Compiler is a program that reads a program written in one language (Source Language like C,C++,etc…) and translate it into an equivalent program in another language (Target Language like Machine Language) and the Compiler reports to its user the presence of errors in the source program.



- **Classification of Compiler:**
  1. Single Pass Compiler (narrow): Traverse the source program in only once. Faster, has limited scope of passes, eg. Pascal.
  2. Multi-Pass Compiler (wide): Processes the source program in several times. Slower, has wide scope of passes, eg. Java.
  3. Load and Go Compiler: Generates machine code and then immediately executes it.
  4. Debugging or Optimizing Compiler: Tries to minimize or maximize some attributes of an executable computer program.

- **Available C Compliers:**
  - There are **over 50 compilers for C. Some popular C compliers below**
  - Turbo C, Borland C, Ansi C, CC, GCC, Visual Studio Code, etc…

Unit – I

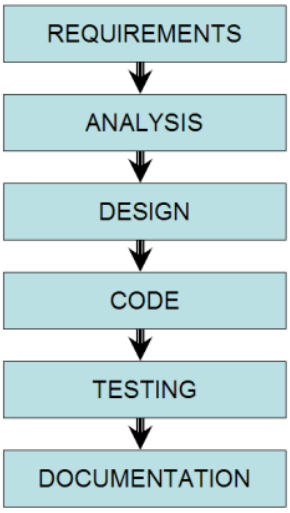
Problem Solving Aspect

- Basic steps to solve the problems:
    - Step 1: Identify and Define Problem
        - Explain your problem clearly as possible as you can.
    - Step 2: Generate Possible Solutions
        - List out all the solution that you find. Don't focus on the quality of the solution
        - Generate the maximum number of solutions as you can without considering the quality of the solution
    - Step 3: Evaluate Alternatives
        - After generating the maximum solution, Remove the undesired solutions.
    - Step 4: Decide a Solution
        - After filtering all the solution, you have the best solution only. Then choose on of the best solution and make a decision to make it as a perfect solution.
    - Step 5: Implement a Solution
        - After getting the best solution, implement that solution to solve a problem.
    - Step 6: Evaluate the result
        - After implementing a best solution, evaluate how much you solution solve the problem. If your solution will not solve the problem then you can again start with Step 2.

Introduction to Problem Solving

- Problem solving is a process of transforming the description of a problem into the solution of that problem by using our knowledge of the problem domain and by relying on our ability to select and use appropriate problem-solving Strategies, Techniques and Tools.
- Problem solving (within the context of developing programs) refers to analyzing a problem with the intention of deriving a solution for the problem.
- Using computers in problem solving.
- Software development method (SDM).
- Consists of the following steps:
    - Requirements specifications (Specification of needs)
        - State the problem clearly and unambiguously (Doubtless) to understand exactly. What the problem is?, What is needed to solve it?, What the solution should provide and If there are constraints and special conditions.
    - Problem analysis
        - In the analysis phase, we should identity the problem, their form and the input media to be used and expected from the problem, their form and the output media to be used. Special constraints or (necessity) conditions (if any). Formulas or equations to be used.
    - Design the algorithm to solve the problem
        - An algorithm is a list of steps to be executed with the right order in which these steps should be executed.
    - Implementation
        - Implement the solution for identified problem using any computer language.

- Testing and verification
  - Apply some sample input and test whether working is properly or not.
- Documentation
  - Create the document all process.



- General problem-solving strategies:
  - Divide and Conquer:
    - The basic idea is to divide the original problem into two or more sub problems which can be solved more efficiently by the same technique.
    - It is possible to proceed with the splitting into smaller and smaller problems we will reach the stage where the sub problems are small enough to be solved without further splitting. In particularly applicable for sorting, selection, etc.
    - It is not necessary to always exactly halve the problem size. Sometimes we need to always apply the divide – and –conquer strategy in reverse to some problems. The resulting is a binary doubling strategy
  - Dynamic Programming:
    - This method is used most often when we have to build up a solution to a problem via sequence of intermediate steps. This method relies on the idea that a good solution to a large problem can sometimes be built up from good or optimal solutions to smaller problems.

Top-Down Design

- Algorithm needed to solve a problem is more complex, then we must break up the problem into sub problems to develop the program solution.
- To solve a sub – problem at one level, we introduce new sub problems at lower levels this process is called top – down design, proceeds from the original problem at the top level to the sub problems at each lower level.
- A problem solving method in which you first break a problem up into its major sub problems to derive the solution to the original problem.
- The splitting of a problem into its related sub problems is same as the process of refining an algorithm.

Unit – I

- Example:
  - To create the stick figure, you can divide the problem into three sub problems
    1. Draw a circle
    2. Draw a triangle
    3. Draw a intersecting lines

    Still step 2 can refine into two sub problems
    - 2.1 Draw intersecting lines
    - 2.2 Draw a base

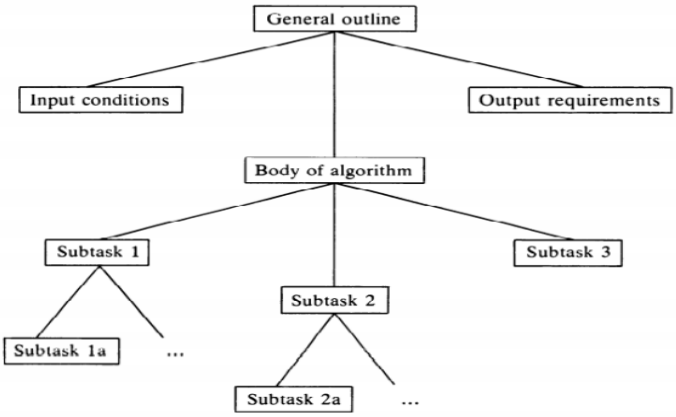


Fig. schematic breakdown of a problem into subtasks as employed in top-down design

Steps to solve logical and numerical problems.

1. Read the problem at least three times (or however many makes you feel comfortable)
2. Work through the problem manually with at least three sets of sample data
3. Simplify and optimize your steps
4. Write pseudocode
5. Translate pseudocode into code and debug
6. Simplify and optimize your code

Algorithm

- A set of finite rules or instructions to be followed in calculations or other problem-solving operations.

Or

- A procedure for solving a mathematical problem in a finite number of steps that frequently involves recursive operations.

Or

- An algorithm is a set of instructions for solving a problem or accomplishing a task

Unit – I

Representation of Algorithm

An algorithm is a step – by – step procedure for solving a particular problem.

- **Important characterstics of an algorithm:**
  - Clear and Unambiguous: The algorithm should be unambiguous. Each of its steps should be clear in all aspects and must lead to only one meaning.
  - Well-Defined Inputs: If an algorithm says to take inputs, it should be well-defined inputs. It may or may not take input.
  - Well-Defined Outputs: The algorithm must clearly define what output will be yielded and it should be well-defined as well. It should produce at least 1 output.
  - Finite-ness: The algorithm must be finite, i.e. it should terminate after a finite time.
  - Feasible: The algorithm must be simple, generic, and practical, such that it can be executed with the available resources. It must not contain some future technology or anything.
  - Language Independent: The Algorithm designed must be language-independent, i.e. it must be just plain instructions that can be implemented in any language, and yet the output will be the same, as expected.

- **Advantages of an algorithm:**
  - It provides the core solution to an given problem.
  - It facilitates problem development by acting as a design document or a blue print of a given problem.
  - It ensures easy comprehension of a program solution as compared to equivalent computer program.
  - It eases identification and removal of logical errors in a program.

- **Disadvantages of an algorithm:**
  - In large algorithm, the flow of program control becomes difficult to trace.
  - Algorithms lack visual representation of programming constructs link flow chart.

- **Example Problems:**
  1. Write an algorithm for adding two integer Number.
     Step 1: Start a program
     Step 2: Declare the variables ‘a’, ‘b’ and ‘c’
     Step 3: Take input from user and store in ‘a’ and ‘b’
     Step 4: Add the values of a and b and store in c
     Step 5: Print c value
     Step 6: End a Program

  2. Write an algorithm for finding area of circle.
     Step 1: Start a program
     Step 2: Declare the variables ‘radius’ and ‘area’
     Step 3: Take input from user and store in ‘radius’
     Step 4: Compute area=3.14*radius*radius
     Step 5: Print area value
     Step 6: End a Program

Unit – I

3. Write an algorithm for finding the greater of two numbers.
   Step 1: Start a program
   Step 2: Declare the variables 'a', and 'b'
   Step 3: Take input from user and store in 'a' and 'b'
   Step 4: Compare a and b; If a is greater then Goto Step 7
   Step 5: Print "b is greater number"
   Step 6: Goto Step 8
   Step 7: Print "a is greater number"
   Step 8: End a Program

4. Write an algorithm for finding the area and circumference of a circle of given radius.
   Step 1: Start a program
   Step 2: Declare the variables 'radius', 'circum' and 'area'
   Step 3: Take input from user and store in 'radius'
   Step 4: Compute circum = 2 * 3.14 * radius
   Step 5: Compute area = 3.14 * radius * radius
   Step 6: Print circum and area values
   Step 7: End a Program

5. Write an algorithm for finding the volume of a sphere of given radius.
   Step 1: Start a program
   Step 2: Declare the variables 'radius', and 'volume'
   Step 3: Take input from user and store in 'radius'
   Step 4: Compute volume= (4/3) * 3.14 * radius * radius * radius
   Step 5: Print volume value
   Step 6: End a Program

6. Write an algorithm for finding the interest on a given principle for a given period of time at a given rate of interest per year.
   Step 1: Start a program
   Step 2: Declare the variables p, n, r and si
   Step 3: Take inputs from user and store in p, n and r
   Step 4: Compute si = (p * n * r) / 100
   Step 5: Print si value
   Step 6: End a Program

Flow charts:

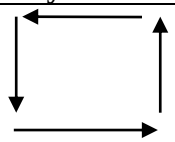
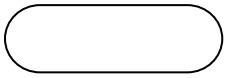
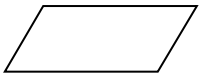
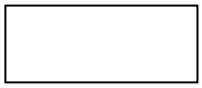
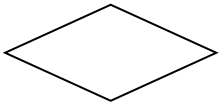
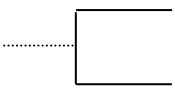
- Definition:
  - A flowchart is a type of diagram that represents a workflow or process.

    Or
  - A flowchart can also be defined as a diagrammatic representation of an algorithm, a step-by-step approach to solving a task.

    Or
  - Flowchart is nothing but the graphical representation of the data or the algorithm for a better understanding of the code visually.

Unit – I

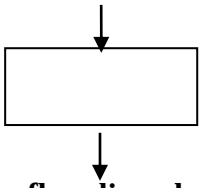
- **Representation of Symbols:**
  - Flowchart is a **pictorial representation** of an algorithm in which the steps are drawn in the form of different shapes of boxes and the **logical flow** is indicated by interconnecting arrows.
  - The **boxes** represent **operations** and the **arrows** represent the **sequence** in which the operations are implemented.
  - Flowchart is drawn according to **defined rules and using standard flowchart symbols** prescribed by ANSI (American National Standard Institute)
- **Some standard symbols are…**

Symbol	Symbol Name	Description
	**Flow Lines**	• Flow lines are used to **connect symbols**. • These lines indicate the **sequence of steps** and the direction of flow of control.
	**Terminals**	• This symbol (rounded rectangle) is used to represent the **Start and Stop operation** in the program logic.
	**Input / Output**	• This symbol (Parallelogram) represent getting **input** (given by the user to the process) and **output** (given by the process to the user)
	**Processing**	• Process symbol is used for representing **arithmetic and data operation**
	**Decision**	• Decision symbol denotes a **decision** to be made. • This symbol has **one entry and two exit paths.** • The ***path depends on the decision process***
	**Connector**	• Connector symbol is used to **join different flow lines**
	**Declare**	• This symbol is used to indicate that the declaration of variables.
	**Document**	• Document is used to represent a **paper document** produced during the flowchart process
	**Annotation**	• It is used to provide **additional information** about flowchart symbol • The content may be in the form of descriptive comments

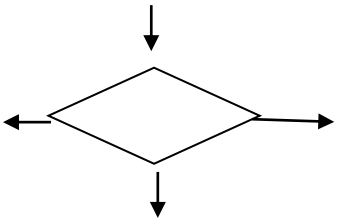
Unit – I

- **Guidelines for preparing flow charts:**

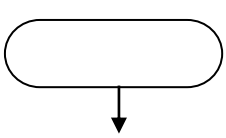
  Following guidelines should be used for creating a flowchart:
  - The flowchart should be **clear, neat and easy** to follow
  - The flowchart must have a **logical start and finish**
  - **Only one flow line** should come out from a **process** symbol

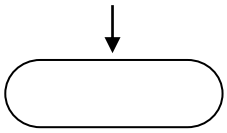


  - **Only one flow line should enter a decision symbol**. However, two or three flow lines may leave the decision symbol

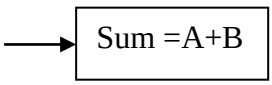


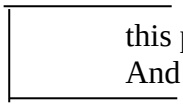
  - Only one flow line is used with a terminal symbol





  - if necessary, use annotation symbol to describe data or process





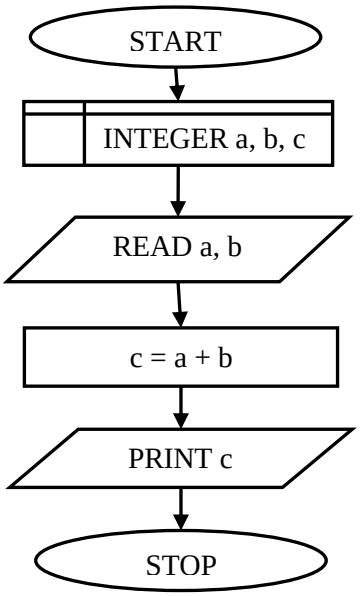
  - In case of complex flowcharts, **connector symbols** are used to reduce the number of lines
  - **Intersection of flow lines** should be **avoided** to make it more effective

- **Benefits or advantage of flowcharts:**
  The reasons for using flowcharts as problem solving tool are given below
  - **Makes logic clear**: The main advantage of using a flow chart to **plan a task**, which makes the logic easier to follow.
  - **Communication:** flowcharts are better way of communicating the logic of a system to all concern.
  - **Useful in coding**: Flowchart acts as a **guide or blueprint** during the analysis and program development phase.

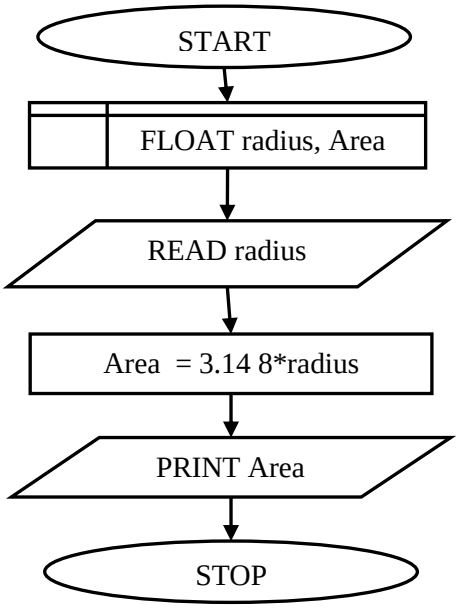
Unit – I

- **Limitation or disadvantage of flowchart:**
  - **Complex:** when a program is very large the flowcharts may continue for many pages, making them hard to follow.
  - **Costly:** If flowchart is to be drawn for a huge application, the time and cost factor of program development may get out of proportion, making it a costly affair.
  - **Difficult to modify:** Due to symbolic nature, any change or modification to a flowchart usually requires redrawing the entire logic again, redrawing a complex flowchart is not a simple task.

- **Example Problems:**
  1. Draw a Flowchart for adding two integer Number.

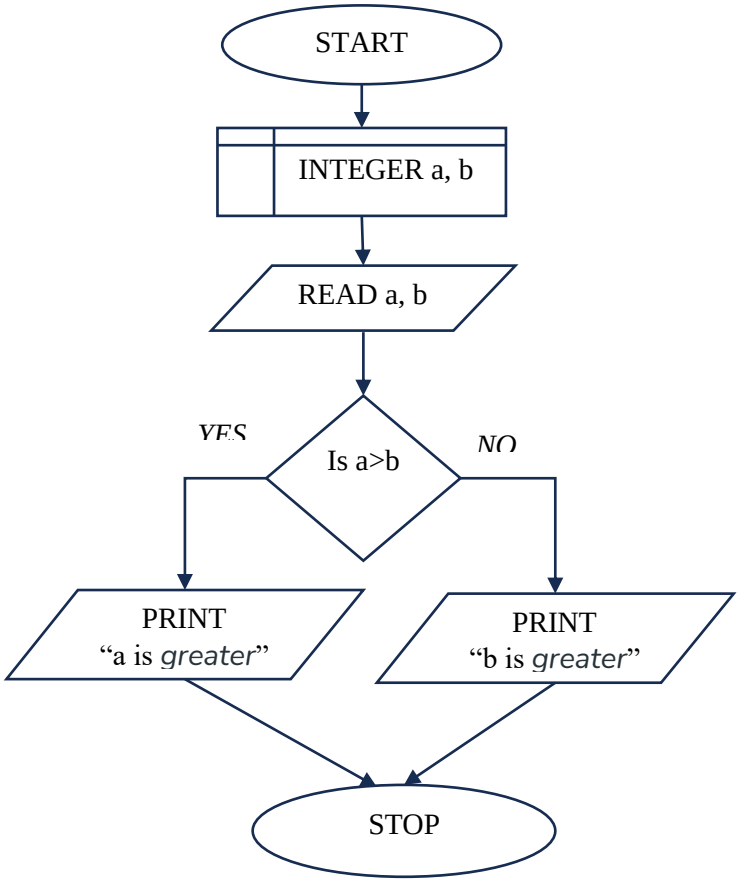


  2. Draw a Flowchart for finding area of circle.

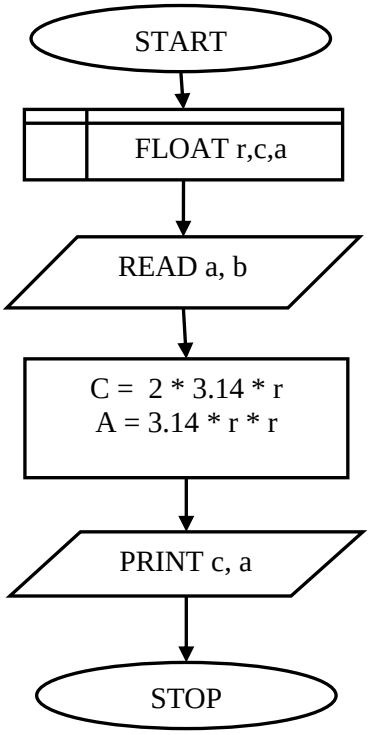


Unit – I

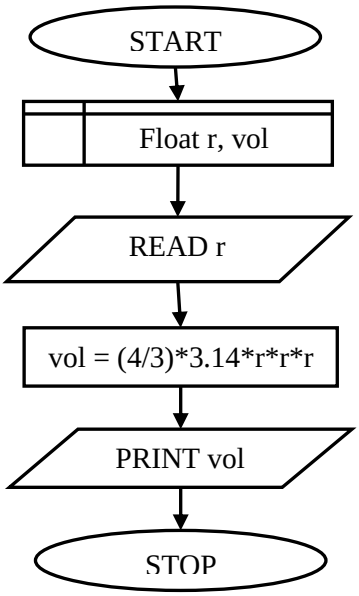
3. Draw a Flowchart for finding the greater of two numbers.



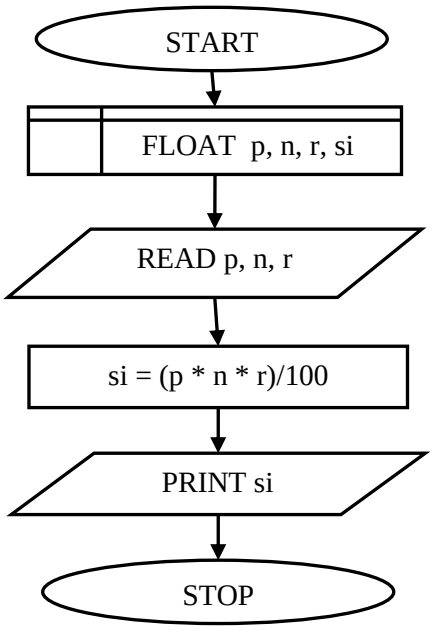
4. Draw a Flowchart for finding the area and circumference of a circle of given radius.



5. Draw a Flowchart for finding the volume of a sphere of given radius.



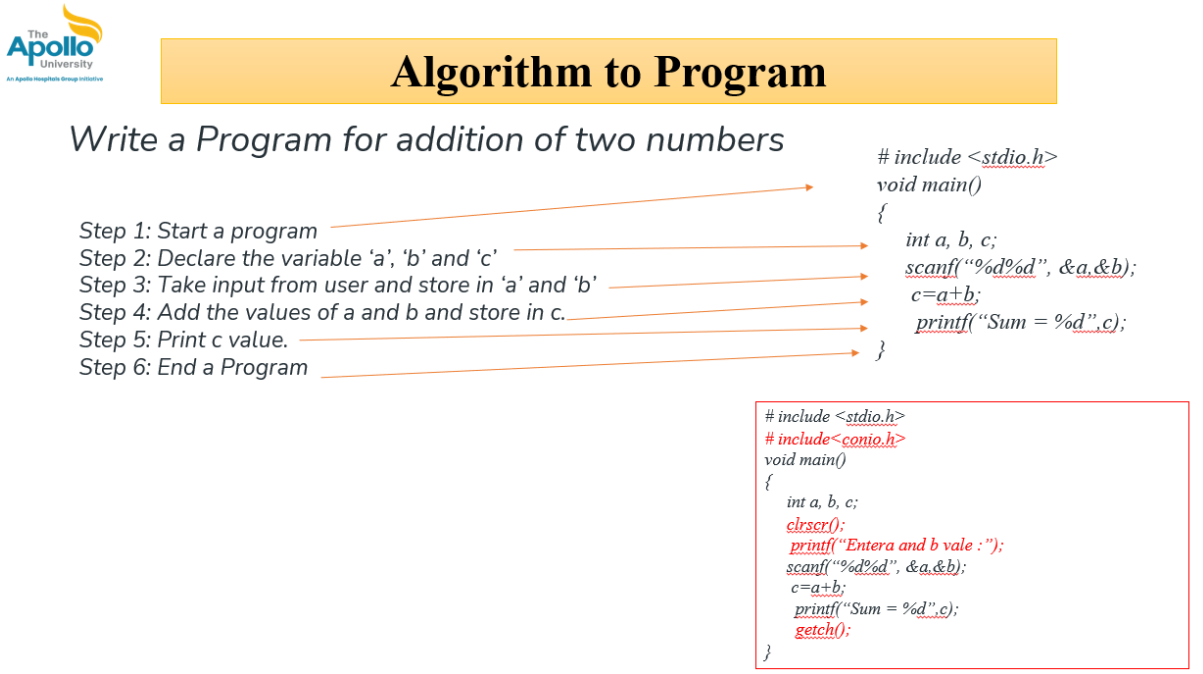
6. Draw a Flowchart for finding the interest on a given principle for a given period of time at a given rate of interest per year.



Source code

- Source code is generally understood to mean programming statements that are created by a programmer with a text editor or a visual programming tool and then saved in a file.

Unit – I



Variables

- A variable is a user-defined or a user-readable custom name assigned to a memory location.
- Variables hold a value that can be modified and reused many times during the program execution.

- **Rules for Naming Variables in C**
  - Names can contain letters, digits and underscores
  - Names must begin with a letter or an underscore (_)
  - Names are case sensitive (Name and name are different variables)
  - Names cannot contain whitespaces or special characters like !, #, %, etc.
  - No keywords will be considered as a variable.
  - The identifier length is limited to 31 characters.

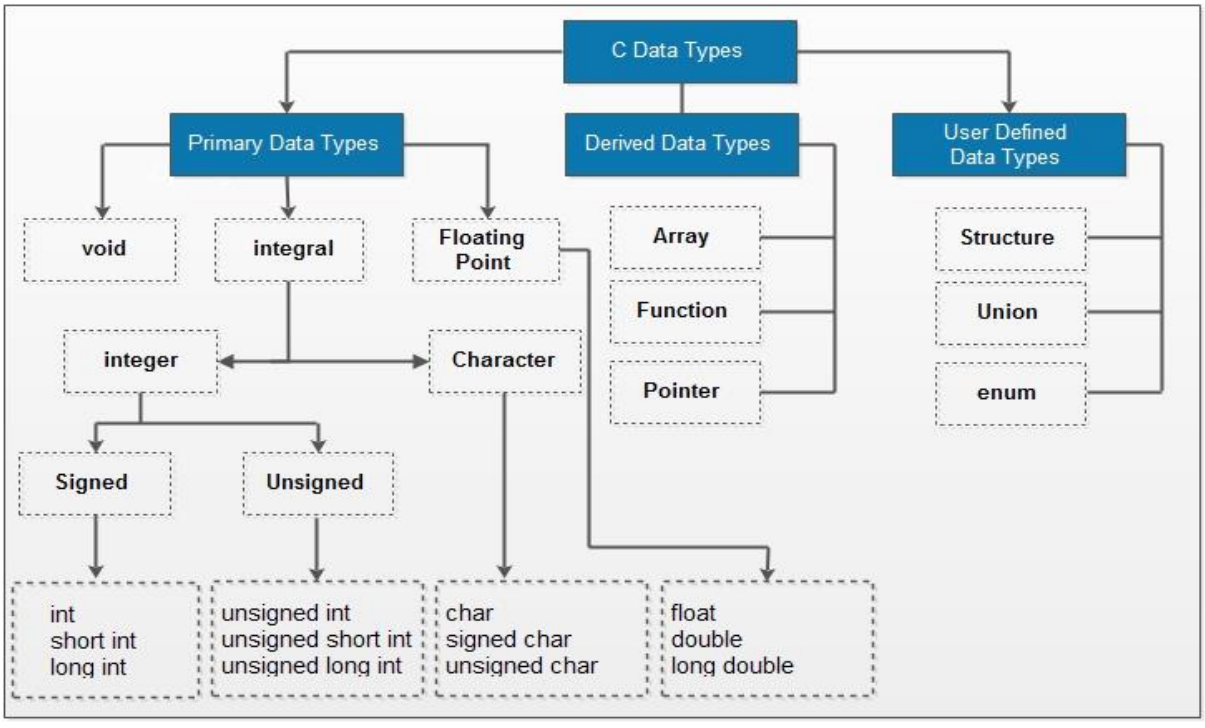
- **Declaration of Variables**
  - Declaration of a variable in a computer programming language is a statement used to specify the variable name and its data type.
  - Declaration tells the compiler about the existence of an entity in the program and its location.
  - A few simple parameters are considered to declare a variable in the C programming language. These parameters are collectively termed syntax.
  - Syntax:
    - <data type> <variable-list>;
  - Example:
    - int a, b, sum;

Unit – I

- **Initialization of Variables**
  - It is the process of assigning a value to the Variable.
  - Every programming language has its own method of initializing the variable.
  - If the value is not assigned to the Variable, compilers store a random or a garbage value in the declared variable.
  - Syntax:
    - <data type> <variable-name = value>;
    - <data type> <var1 = val1>, <var2 = val2>,.....<varn = valn>;
  - Example:
    - int a = 10, b=20,c;

Datatypes

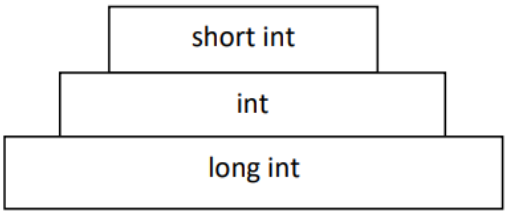
- Each variable in C has an associated data type.
- It specifies the type of data that the variable can store like integer, character, floating, double, etc.
- Each data type requires different amounts of memory.
- There three types of datatypes.
  - Primitive Data Types
    - Primitive data types are the most basic data types that are used for representing simple values such as integers, float, characters, etc.
  - User Defined Data Types
    - The user-defined data types are defined by the user himself.
  - Derived Types
    - The data types that are derived from the primitive or built-in datatypes are referred to as Derived Data Types.



Unit – I

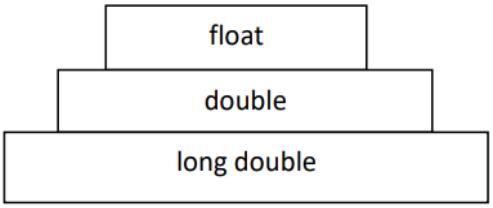
Integer types:

- Integers are whole numbers with a range of values supported by a particular machine
- Integer occupy one word of storage
- Word size of machines vary (16 or 32 bits)
- If we use a 16 bit, the range is -32768 to 32767 (that is, -2^{15} to $+2^{15}$ -1)
- A signed integer uses one bit for sign and 15 bits for the magnitude of the number
- 32 bit store an integer ranging from -2,147,483,648 to 2,147,483,647
- C has three classes of integer storage
  - short
  - int
  - long int in both signed and unsigned forms
- Short int represents small integer and requires half the amount of storage as a regular int
- Unsigned integers use all the bits for the magnitudes the number and are always positive then in 16 bit the range is 0 to 65535
- Default declaration is a signed number



Floating point types:

- Floating point(or real) numbers are stored in 32 bits (on all 16 or 32 bit machines), with 6 digits precision
- Floating point number are defined in C by the keyword float
- When the accuracy provided by float is not sufficient the type double used
- A double data type uses 64 bits giving precision of 14 digits
- These are known as double precision number
- To extend the precision further, we may use long double which uses 80 bits



Character types:

- A single character can be defined as a character(char) type data
- Characters are usually stored in 8 bits (one byte) of internal storage
- Unsigned chars have values between 0 and 255, signed chars have values from -128 to 127

Unit – I

Void type:

- The void type has no values
- It is usually used to specify the type of functions
- The type of a function is said to be void when it does not return any value to the calling function

Table: size and range of data types on a 16 – bit machine

Type	Size		Range
	Bit	bytes	
char or signed char	8	1	-128 to 127
unsigned char	8	1	0 to 255
int or signed int	16	2	-32768 to 32767
unsigned int	16	2	0 to 65535
short int or signed short int	8	1	-128 to 127
unsigned short int	8	1	0 to 255
long int or signed long int	32	4	-2,147,483,648 to 2,147,483,647
unsigned long int	32	4	0 to 4,294,967,295
Float	32	4	3.4E-38 to 3.4E+38
Double	64	8	1.7E-308 to 1.7E+308
Long double	80	10	3.4E-4932 to 1.1E+4932

User defined type declaration:

There are two types of user – defined data types

- o Type definition
- o Enumerated

Type definition:

- It allows user to define an identifier that would represent an existing data type
- It can later be used to declare variables typedef type identifier; type refers to an existing data type and identifier refers to the new name given to the data type
- The new type is new only in name, but not the data type
- Typedef cannot create a new type
- Ex: typedef int marks; marks name1, name2;
- By this it increases the readability of the program

Enumerated data type:

- It is used to declare variables that can have one of the values enclosed within the braces (known as enumeration constants)
- After this definition, we can declare variables of this new type
- It is defined as follows:

  enum identifier{value1,value2,value3,…….,valuen};

  enum identifier v1,v2,v3,……..,vn;

- The variables v1,v2,…….,vn can have one of the values value1,value2,…….. v1=value3; v5=value1;
- Example: enum day{mon,tue,wed,thu,fri,sat,sun};
- The compiler automatically assigns integer digits with 0 to all enum constants
- That is enum constant value1 is assigned 0, value2 is assigned 1 and so on

Memory locations

- When a variable is created in C, a memory address is assigned to the variable. The memory address is the location of where the variable is stored on the computer. When we assign a value to the variable, it is stored in this memory address. To access it, use the reference operator (&), and the result will represent where the variable is stored:
- **Example:**

```
int myAge= 43;
printf("%p", &myAge);
```

Syntax and Logical Errors

- An error in the C language is an issue that arises in a program, making the program not work in the way it was supposed to work or may stop it from compiling as well. If an error appears in a program, the program can do one of the following three things: the code will not compile, the program will stop working during execution, or the program will generate garbage values or incorrect output.
- Syntax Error
  - Syntax errors occur when a programmer makes mistakes in typing the code's syntax correctly or makes typos. In other words, syntax errors occur when a programmer does not follow the set of rules defined for the syntax of C language.
  - Syntax errors are sometimes also called compilation errors because they are always detected by the compiler. Generally, these errors can be easily identified and rectified by programmers.
  - The most commonly occurring syntax errors in C language are:
    - Missing semi-colon (;)
    - Missing parenthesis ({})
    - Assigning value to a variable without declaring it
  - Example 1:

```
#include <stdio.h>
void main() {
   var = 5;    // we did not declare the data type of variable
   printf("The variable is: %d", var);
}
```

Output:

error: 'var' undeclared (first use in this function)

If the user assigns any value to a variable without defining the data type of the variable, the compiler throws a syntax error.

- Example 2:

```
#include <stdio.h>
void main() {
   for (int i = 0;) {  // incorrect syntax of the for loop
      printf("Scaler Academy");
   }
}
```

Output:

error: expected expression before ')' token

A *for* loop needs 3 arguments to run. Since we entered only one argument, the compiler threw a syntax error.

- **Logical Error**
  - Sometimes, we do not get the output we expected after the compilation and execution of a program. Even though the code seems error free, the output generated is different from the expected one. These types of errors are called Logical Errors. Logical errors are those errors in which we think that our code is correct, the code compiles without any error and gives no error while it is running, but the output we get is different from the output we expected.
  - In 1999, NASA lost a spacecraft due to a logical error. This happened because of some miscalculations between the English and the American Units. The software was coded to work for one system but was used with the other.
  - Example:

```
#include <stdio.h>
void main() {
   float a = 10;
   float b = 5;
   if (b = 0) {  // we wrote = instead of ==
      printf("Division by zero is not possible");
   }
   else {
      printf("The output is: %f", a/b);
   }
}
```

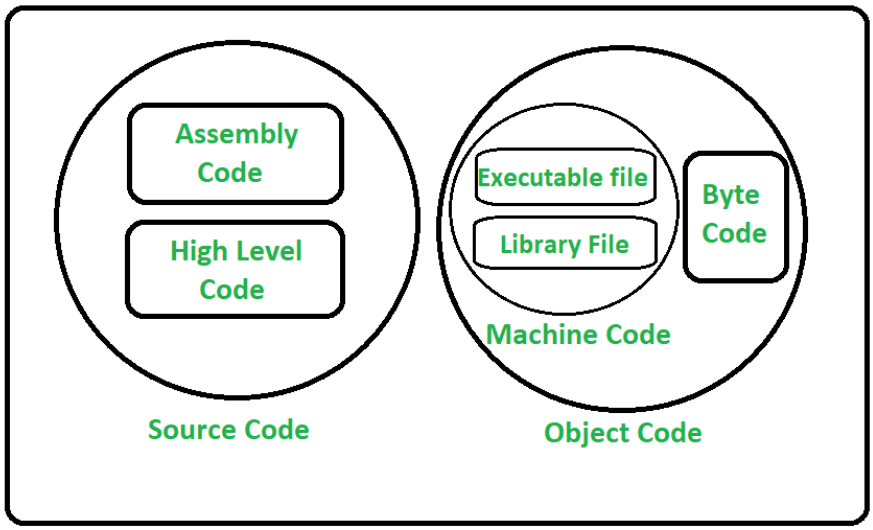
Output:

The output is: inf

INF signifies a division by zero error. In the above example, at line 8, we wanted to check whether the variable b was equal to zero. But instead of using the equal to comparison operator (==), we use the assignment operator (=). Because of this, the if statement became false and the value of b became 0. Finally, the else clause got executed.

Unit – I

Object and Executable code:



- Object code generally refers to the output, a compiled file, which is produced when the Source Code is compiled with a C compiler. The object code file contains a sequence of machine-readable instructions that is processed by the CPU in a computer.

- Executable code generally refers to machine language, which is the set of native instructions the computer carries out in hardware. Executable Code means the fully compiled version of a software program that can be executed by a computer and used by an end user without further compilation.
- **Here are some similarities between source code and object code:**
    - Both are necessary for the software development process: Source code is used to create software, and object code is the final product of the compilation process that is executed by the computer.
    - Both are created using programming languages: Source code is written in a high-level programming language such as Java, C++, or Python, and object code is generated by the compiler when it translates the source code into machine code.
    - Both are necessary for debugging: Source code is used for debugging during the development process, while object code can be used for debugging after the software has been compiled.
    - Both are necessary for software maintenance: Source code is used for making changes or modifications to the software, and object code can be used for troubleshooting and resolving issues that may arise during the software's lifespan.
    - Both can be protected by copyright law: Source code and object code can be protected by copyright law to prevent unauthorized copying or use.